

Задание №1

а) Напишите класс Ball, который должен иметь следующие две закрытые переменные-члены со значениями по умолчанию:

- `m_color (Red);`
- `m_radius (20.0).`

В классе Ball должны быть следующие конструкторы:

- для установления значения только для `m_color`;
- для установления значения только для `m_radius`;
- для установления значений и для `m_radius`, и для `m_color`;
- для установления значений, когда значения не предоставлены вообще.

Не используйте параметры по умолчанию для конструкторов. Напишите еще одну функцию для вывода цвета (`m_color`) и радиуса (`m_radius`) шара (объекта класса Ball).

Следующий код функции main():

```
1. int main()
2. {
3.     Ball def;
4.     def.print();
5.
6.     Ball black("black");
7.
8.     black.print();
9.
10.    Ball thirty(30.0);
11.    thirty.print();
12.
13.    Ball blackThirty("black", 30.0);
14.    blackThirty.print();
15.
16.    return 0;
17. }
```

Должен выдавать следующий результат:

```
color: red, radius: 20
color: black, radius: 20
color: red, radius: 30
color: black, radius: 30
```

б) Теперь обновите ваш код из предыдущего задания с использованием конструкторов с параметрами по умолчанию. Постарайтесь использовать как можно меньше конструкторов.

Задание №2

а) Напишите простой класс с именем Numbers. Этот класс должен иметь:

- три закрытые переменные-члены типа double: `m_a`, `m_b` и `m_c`;
- открытый метод с именем `setValues()`, который позволит устанавливать значения для `m_a`, `m_b` и `m_c`;
- открытый метод с именем `print()`, который будет выводить объект класса Numbers в следующем формате: `<m_a, m_b, m_c>`.

Следующий код функции `main()`:

```
1. int main()
2. {
3.     Numbers point;
4.     point.setValues(3.0, 4.0, 5.0);
5.
6.     point.print();
7.
8.     return 0;
9. }
```

Должен выдавать следующий результат:

```
<3, 4, 5>
```

б) Добавьте функцию `isEqual()` в класс Numbers, чтобы следующий код работал корректно:

```
1. int main()
2. {
3.     Numbers point1;
4.     point1.setValues(3.0, 4.0, 5.0);
5.
6.     Numbers point2;
7.     point2.setValues(3.0, 4.0, 5.0);
8.
9.     if (point1.isEqual(point2))
10.        std::cout << "point1 and point2 are equal\n";
11.     else
12.        std::cout << "point1 and point2 are not equal\n";
13.
14.     Numbers point3;
15.     point3.setValues(7.0, 8.0, 9.0);
16.
17.     if (point1.isEqual(point3))
18.        std::cout << "point1 and point3 are equal\n";
19.     else
20.        std::cout << "point1 and point3 are not equal\n";
21.
22.     return 0;
}
```

Задание №3

Теперь попробуем что-то посложнее. Напишите класс, который реализует функционал стека. Класс Stack должен иметь:

- закрытый целочисленный фиксированный массив длиной 10 элементов;
- закрытое целочисленное значение для отслеживания длины стека;
- открытый метод с именем `reset()`, который будет инициализировать значением `0` длину и все значения элементов;
- открытый метод с именем `push()`, который будет добавлять значение в стек. Метод `push()` должен возвращать значение `false`, если массив уже заполнен, в противном случае — `true`;
- открытый метод с именем `pop()` для возврата значений из стека. Если в стеке нет значений, то должен выводиться стейтмент `assert`;
- открытый метод с именем `print()`, который будет выводить все значения стека.

Следующий код функции `main()`:

```
1. int main()
2. {
3.     Stack stack;
4.     stack.reset();
5.
6.     stack.print();
7.
8.     stack.push(3);
9.     stack.push(7);
10.    stack.push(5);
11.    stack.print();
12.
13.    stack.pop();
14.    stack.print();
15.
16.    stack.pop();
17.    stack.pop();
18.
19.    stack.print();
20.
21.    return 0;
22. }
```

Должен выдавать следующий результат:

```
( )
( 3 7 5 )
( 3 7 )
( )
```